UNIVERSITY OF CENTRAL FLORIDA (UCF)

Department of Computer Science

COP 3502 Computer Science I

Common Midterm Examination Spring 2026

Total points: 100 Total Time: 2 hours

COP 3502 Sections *(Circle your section)*	**Instructor**
Circle one: Sec 1 or Sec 206 (H)	Mahfuzur Rahaman
Circle one: Sec 2 or Sec 201 (H)	Tanvir Ahmed
Circle one: Sec 3 or Sec 4 or Sec 5	**Yancy Vance Paredes**
Circle one: Sec 203 (H) or Sec 205(H)	Kurt Kullu
Circle: Section 204 (H)	Awrad Mohammed Ali

Last Name in Upper Case Letter: _____

First Name in Upper Case Letter: _____

UCF ID: _____

Did you circle your section above? _____

Instructions:

1. This is a **closed-book** and **closed-neighbor** exam. No notes, textbooks, or outside assistance are permitted.
2. **Calculators, smartwatches, and electronic devices** (phones, tablets, laptops, earbuds, etc.) are strictly prohibited.
3. **No scratch paper** is allowed. However, you may use the margins or back side of the exam paper for rough work.
4. Write **clearly and legibly**. If the instructor cannot read your handwriting, the answer may not receive credit.
5. Manage your time wisely

A. Dynamic Memory Allocation [Total: (3 + 10 + 7) = 20 pts]

1. Suppose an int pointer called **`arr`** is pointing to a dynamically allocated int array of size **`n`**. However, the array is completely full. Write a line of code that can double the size of the array without losing any data.
   Answer:

2. The definition of a student structure is provided below:

```
typedef struct Student_s {
    char *name;
    int score;
} Student;
```

Complete the function that accepts an array of student names, an array of their corresponding scores, and the number of students **`n`**. You may assume that **`names[i]`** corresponds to the student whose score is **`scores[i]`**, and that both arrays are non-empty. The function should create and return a dynamically allocated array of **`n`** pointers to **`Student`** structures. Each pointer should reference a separately dynamically allocated **`Student`** structure, and each student's name must also be dynamically allocated and copied from the names array. The function must allocate the exact space needed to store the strings.

```
Student **createRoster(char names[][101], int scores[], int n) {
```

3. A cat shelter keeps track of cats and their toys. Each cat has a **name stored directly in the struct** and a list of toy names stored as a dynamically allocated array of strings. Consider the given struct:

```
typedef struct cat_toys {
    char name[50];   // cat name
    char **toys;     // dynamically allocated array of toy names
    int numToys;     // number of toys
} cat_toys;
```

Write a function that takes a pointer to a dynamically allocated cat_toys struct and frees the struct and related memory without creating any memory leak.

```
void erase_cat_toys(cat_toys *c) {
```

B. Recursion [Total: (7 + 6 + 7 = 20 pts)]

1. The following function calculates the sum of the digits of an integer using iteration.

```
int sumDigits(int n) {
    int sum = 0;

    while (n != 0) {
        sum += n % 10;
        n /= 10;
    }

    return sum;
}
```

Rewrite this function recursively so that it produces the same result.

```
int sumDigits(int n) {
```

2. Consider the following node struct for a singly linked list.
   `typedef struct node { int data;  struct node* next; } node;`
   Write a recursive function **`printRev(node *head)`**, that receives the head of a singly linked list and prints all the **even numbers** in the list in reverse order. (Note that you are not reversing the linked list. You are just printing the list in reverse order.
   For example, for the list 3 → 2 → 6 → 1 → 8 → 4 The function should print: **`4 8 6 2`**

```
void printRev(node *head) {
```

3. Three friends {"Hana", "Coco", "Petra"} are going to watch a cat competition. There are 6 possible seating arrangements for them.

   - ☐ Hana, Coco, Petra
   - ☐ Hana, Petra, Coco
   - ☐ Coco, Hana, Petra
   - ☐ Coco, Petra, Hana
   - ☐ Petra, Hana, Coco
   - ☐ Petra, Coco, Hana

We need to print all seating arrangements using permutations. Our permutation algorithm takes the number of friends **`n`** as a parameter and recursively generates all permutations of the numbers from **`0 to n-1`**. Complete the following code using the **"used" array technique** you learned in class, and then complete the **`print()`** function to display the seating arrangements based on the permutation generated by the **`makePerm()`** function.

Fill in **`makePerm()`** function and **the `print()` function** based on the above requirement.

```
#define SIZE 3
void makePerm(int* perm, int* used, int k, int n);
void print(int* perm, int n);

char friends[SIZE][50] = {"Hana", "Coco", "Petra"}; // array of friends name


int main() {
    int perm[SIZE] = {0};
    int used[SIZE] = {0};
    makePerm(perm, used, 0, SIZE);
    return 0;
}

void makePerm(int* perm, int* used, int k, int n) {

    if (__________________) {
        print(perm, n);
        return;
    }
/*** FILL IN FOR LOOP ***/
    for (int i=0; i<n; i++) {
        if (!used[i]) {
            used[___] = _____;

            perm[_____] = _____;

            makePerm(_______, _________, _________, __________);

            used[_______] = _______;
      }
    }
}  // this question continues to the next page.
```

```
// Write the print function to print the friend's name based on the permutation.
// Print all the names of that permutation in a single line, separated by a space

void print(int* perm, int n) {
```

C. Linked List [10 + 8 = 18 pts]

1. Consider a typical **doubly linked list** node struct.

```
typedef struct node {
    int data;
    struct node* prev;
    struct node* next;
} node;
```

Write a **non-recursive function** that takes the head of a linked list containing unique numbers and an integer x. The function should delete the node containing x (if it exists) **without causing a memory leak**, and return the head of the linked list. If x is not present, the list should remain unchanged.

Example: If the linked list is **`10 -> 20 -> 30 -> 7 -> 4 -> 8`** and **`x = 7`**, the resulting list should be **`10 -> 20 -> 30 -> 4 -> 8`**. The function should return the original head in this case, since the head node was not deleted. However, if **`x = 10`**, the function should return the updated head.

```
node* deleteXdoubly(node* head, int x) { // Write this function
```

2. Consider head is a node pointer pointing to the first node of the linked list.

   $head \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow NULL$

   Show the status of the complete linked list after the following function call:
   `head = meow(head);`
   Please list the items in the same format as shown above, with arrows.

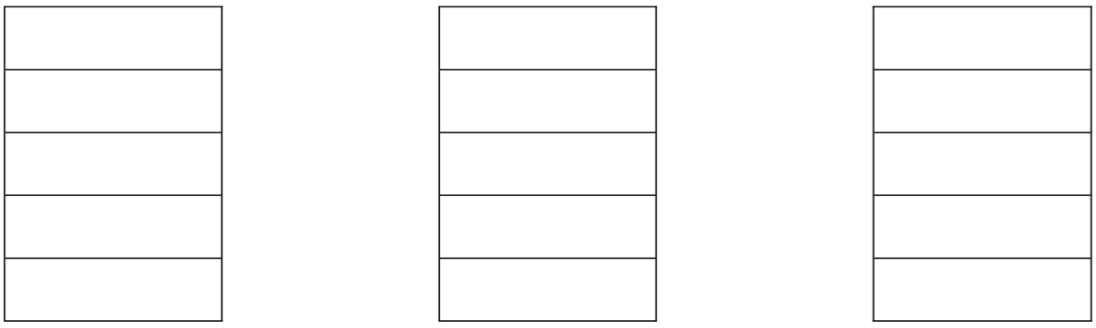
```
typedef struct node {
    int data;
    struct node *next;
} node;

node* meow(node* head) {
    if (head == NULL || head->next == NULL) return head;

    node *cur = head;
    while (cur != NULL && cur->next != NULL) {
        node *temp = cur->next;
        cur->next = temp->next;
        temp->next = head;
        head = temp;
        cur = cur->next;
    }

    return head;
}
```

Answer:

D. Stack and Queues [Grade: 10 + 5 + 5 = 20 pts]

1. Evaluate the following postfix expression shown below, using the algorithm that utilizes an operand stack. Show the content of the operand stack at each of the indicated points in the expression. It means whenever you reach point A during the evaluation process, the status of the operand stack at point A should be shown in the stack marked with A. Similarly, the stack marked with B should show the status at point B, and so on.

3 4 2 1 + * 24 Ⓐ 6 / 9 Ⓑ 4 - Ⓒ + - /



A B C

Final Evaluation of the Postfix Expression: _____

2. A **circular queue** of capacity 5 has the following properties:

```
int arr[5];
int capacity = 5;
front = 3;
rear = 0;  // represents the index of the rear (back) of the queue
Qsize = 3; // represents the number of items in the queue
```

The current status of the queue is the following:

Index:	0	1	2	3	4
Value:	20			40	10

We have a list of operations to perform in this queue. For every step, show the array, the front and rear values, and whether the operation succeeded or failed. Fill in the following table to answer this question:

Operation	**front**	**rear**	**Qsize**	**Queue status**	**Success/Fail**
ENQUEUE(50)				0 1 2 3 4	
DEQUEUE()				0 1 2 3 4	
ENQUEUE(60)				0 1 2 3 4	
ENQUEUE(70)				0 1 2 3 4	
ENQUEUE(80)				0 1 2 3 4	

3. A linked list-based queue is implemented with the following structures:

```
// Stores one node of the linked list.
struct node {
   int data;
   struct node* next;
};
```

```
// Stores our queue.
struct queue {
   struct node* front;
   struct node* rear;
};
```

Initially, both the front and rear are initialized to NULL. Answer the following two questions:

a) Is there a situation where we need to update the front of the queue while performing enqueue? Justify your answer with one or two sentences. Just writing yes/no will not receive any credit.

Answer:

b) Is there a situation where we need to update the rear of the queue while performing the dequeue operation? Justify your answer with one or two sentences. Just writing yes/no will not receive any credit.

Answer:

E. Algorithm Analysis [4 + (1 + 2 +2) + 4 + 5 + 4 = 22 pts]

1. You are given two algorithms that solve the same problem: (a) Algorithm A that runs in $\boldsymbol{O(n^2)}$ time, and (b) Algorithm B that runs in $\boldsymbol{O(n \log_5 n)}$ time. However, both algorithms take 25 milliseconds to execute for an input of size 5. For an input size of 125, which algorithm will finish executing first? How much time are we going to save with this algorithm? You must show the steps of your calculations to receive full credit. (Grade: 4 pts)

2. What is the **Big-O** run-time for the following functions? Just write the Big-O. You don't have to explain them. (Grade: 1 + 2 + 2 = 5 pts)

```
void printPairs(int arr[], int n) {

  for (int i = 0; i < n; i++) {

    for (int j = 0; j < n-i; j++) {

      printf("(%d,%d)",arr[i],arr[j]);
    }
  }
}
```

Run-time:

```
void mystery(int n) {
  int i, j, k;
  for (i = 1; i <= n; i++) {
    for (j = 1; j <= n; j *= 2) {
      for (k = 0; k < n; k++) {
        printf("%d %d %d\n", i, j, k);
      }
    }
  }
}
```

Run-time:

```
void analyzeBigOh(int m) {
    int sum = 0;
    while (m>0) {
        sum += m;
        m = m/2;
    }
    return sum;
}
```

Run-time:

3. A developer wrote the following code while implementing the Binary Search algorithm:

```
int BinarySearch(int arr[], int n, int target){
    int low = 0;
    int high = n - 1;
    int mid;
    while(low < high){
        mid = (low + high) / 2;
        if (arr[mid] == target)
            return mid;
        else if (arr[mid] < target)
            low = mid;
        else
            high = mid;
    }
    return -1;
}
```

The developer was using the following inputs while testing the code:

```
arr = {10, 20, 30, 40, 50, 60, 70, 80};
n = 8;
target = 80;
```

Trace the code and show the values of **`low, high, mid, and arr[mid]`** until you either find the target or determine that the item cannot be found, or detect a bug in the code. If you find a bug based on your tracing, then mention what the issue is while processing the task. Then fix and update all the bugs by updating the line(s) in the above code by writing the correction as a comment next to the line(s). ***After correcting the code, you don't need to continue the tracing.*** You may add more rows to the following table if needed. **(Grade: 4 pts)**

low	high	mid	arr[mid]

4. Short answer questions: (Grade: 5 pts)

a) Does an algorithm exist that computes the sum of the integers from `1` to `n` (inclusive) in `O(1)` time? Justify your answer. Just writing yes/no has no credit
Answer:

b) Circle the option: Which of the following Big-O expressions would result in a different Big-O than the others?

i) $2n^2 + 2n + 2$ ii) n^2 iii) $2n^{5/2} + 2$ iv) $100n^2 + 10^6$

c) Put the following Big-O running times in order from **best to worst**.

$$O(n^2), O(\log n), O(1), O(2^n), O(n \log n), O(n^3), O(n)$$

_____, _____, _____, _____, _____, _____, _____
Best Worst

d) What would be the worst-case Big-O runtime to get the 3rd smallest items from a sorted linked list with n items

Answer: _____

e) What would be the best-case run-time to insert an item to the end of a linked list with n items that does not have a tail pointer:

Answer: _____

5. Solve the following summation. You have to show the steps of shifting/splitting to receive full credit: (Grade: 4 pts)

$$\sum_{k=1}^{n} (k-1)$$